Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique

TP ALGO3

Support TP programmation C - les arbres binaires -

Fait par :

Nom et prénom : ABDELMALEK BENMEZIANE

Contents

1 Les arbres binaires en C **1**

1.1 Définition et concepts de base 1

1.2 La structure d'un nœud 1

1.3 Les types d'arbres binaires 1

1.4 Les opérations principales 2

1.5 Parcours d'un arbre 2

1.6 Applications 2

1.7 Avantages et inconvénients 3

1.7.1 Avantages 3

1.7.2 Inconvénients 3

1.8 Implémentation des opérations en langage C 3

1.9 Conclusion 6

Chapter 1

Les arbres binaires en C

1.1 Définition et concepts de base

Un arbre binaire est une structure de données hiérarchique composée de nœuds. Chaque nœud possède au maximum deux enfants :

- un fils gauche
- un fils droit

Chaque sous-arbre est lui-même un arbre binaire, ce qui rend la structure naturellement récursive.

1.2 La structure d'un nœud

Un nœud d'arbre binaire contient généralement une valeur ainsi que deux pointeurs vers ses enfants. Les arbres binaires sont souvent utilisés pour représenter des données hiérarchiques ou pour construire des structures efficaces de recherche.

1.3 Les types d'arbres binaires

- **Arbre binaire simple** : aucune propriété particulière.
- **Arbre binaire de recherche (BST)** : chaque valeur du sous-arbre

gauche est strictement inférieure à la racine, et chaque valeur du sous-arbre droit est strictement supérieure.

- **Arbres complets, parfaits, pleins** : contraintes sur la structure.
- **Arbres équilibrés (AVL, Red-Black)** : garantissent des performances optimales.

1.4 Les opérations principales

Les opérations les plus courantes sur les arbres binaires sont :

- l'insertion d'un nouveau nœud ;
- la recherche d'une valeur ;
- la suppression (optionnelle, plus complexe) ;
- le parcours de l'arbre (préordre, en ordre, postordre).

1.5 Parcours d'un arbre

Les parcours permettent de visiter tous les nœuds d'un arbre selon différents ordres :

- **In-order** : gauche – racine – droite.
- **Pre-order** : racine – gauche – droite.
- **Post-order** : gauche – droite – racine.

1.6 Applications

Les arbres binaires sont utilisés pour :

- organiser des données hiérarchiques ;
- implémenter des arbres de recherche ;
- gérer des structures d'indexation ;

- construire des arbres syntaxiques en compilation.

1.7 Avantages et inconvénients

1.7.1 Avantages

- opérations efficaces dans un arbre équilibré : complexité en $O(\log n)$.
- structure simple et récursive.

1.7.2 Inconvénients

- performances dégradées si l'arbre se dégénère en liste.
- gestion manuelle de la mémoire en C.

1.8 Implémentation des opérations en langage C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *left;
    struct Node *right;
} Node;

Node* createNode(int value);
Node* insert(Node* root, int value);
Node* search(Node* root, int value);
void inorder(Node* root);
void preorder(Node* root);
void postorder(Node* root);
void freeTree(Node* root);


int main() {

    Node* root = NULL;
```

```
    root = insert(root, 8);
    root = insert(root, 3);
    root = insert(root, 10);
    root = insert(root, 1);
    root = insert(root, 6);

    printf("Parcours In-order : ");
    inorder(root);
    printf("\n");

    printf("Parcours Pre-order : ");
    preorder(root);
    printf("\n");

    printf("Parcours Post-order : ");
    postorder(root);
    printf("\n");

    freeTree(root);
    return 0;
}

Node* createNode(int value) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->value = value;
    node->left = NULL;
    node->right = NULL;
    return node;
}

Node* insert(Node* root, int value) {
    if (root == NULL) return createNode(value);
    if (value < root->value)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);
    return root;
}

```

```
Node* search(Node* root, int value) {
    if (root == NULL || root->value == value)
        return root;
    if (value < root->value)
        return search(root->left, value);
    return search(root->right, value);
}

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->value);
    inorder(root->right);
}

void preorder(Node* root) {
    if (!root) return;
    printf("%d ", root->value);
    preorder(root->left);
    preorder(root->right);
}

void postorder(Node* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->value);
}

void freeTree(Node* root) {
    if (!root) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}
```

1.9 Conclusion

Les arbres binaires constituent une structure de données fondamentale en informatique. Grâce à leur organisation hiérarchique et à leur nature récursive, ils permettent de réaliser des opérations de recherche, d'insertion et de parcours de manière efficace, en particulier lorsqu'ils sont équilibrés.

Ils offrent une grande souplesse d'utilisation et se retrouvent dans de nombreux domaines, tels que les algorithmes de recherche, les bases de données, les compilateurs ou encore l'intelligence artificielle.

Cependant, leur efficacité dépend fortement de leur structure, et une mauvaise organisation peut rapidement dégrader les performances. La gestion manuelle de la mémoire en C impose également une rigueur particulière lors de leur utilisation.

En résumé, les arbres binaires sont un outil puissant et polyvalent, dont la compréhension est essentielle pour la maîtrise des structures de données avancées.